Autonomous Drone with Vision-Based Object Tracking

Thy Dinh Dylan Hinton Varun Mange Thomas Nguyen

Abstract

This project attempts to create an autonomous drone that can track and follow a person using onboard computer vision. The system employs a two-computer architecture: a flight controller running ArduPilot for stable flight and a companion computer running a vision pipeline based on YOLO for human detection and DeepSORT for object detection. The tracked object's position in the camera frame is converted into movement commands transmitted to the flight controller via MAVLink, allowing the drone to follow the subject in real time. The initial experiments show accurate human detection and stable tracking, with future work focusing on model fine-tuning, simulation testing, and real-world flight validation.

1. Introduction

The rapid advancement of Unmanned Aerial Vehicles (UAVs) has led to a technological divide. Commercial industry leaders, such as DJI, provide highly sophisticated autonomous features including real-time object tracking and obstacle avoidance. However, these systems are built on proprietary, closed-source architectures, restricting researchers and developers from customizing or auditing the underlying logic. While the open-source community has produced robust flight control firmware like ArduPilot and Betaflight, these platforms do not include vision-based object tracking systems.

This project aims to fill the void by developing a completely **autonomous drone** that performs vision-based object tracking entirely on board, with no need for proprietary software. Our device has a two-computer architecture: a flight controller running **ArduPilot** ensures stable flight, while a companion computer, the Radxa Dragon Q6A, runs a computer vision pipeline that detects and follows a target person in real time. **YOLO11** detects humans in each camera frame in a single neural network pass, while **DeepSORT** maintains a stable identity for the tracked subject across frames, even with occlusions.

2. Related Work

The object detection model **You Only Look Once (YOLO)** is the backbone of our computer vision pipeline. YOLO facilitates quick object localization and classification through its feature extraction and single-pass architecture [7]. Although YOLO is one of the most effective object detection models, it does not inherently maintain object identities across frames. To facilitate object tracking, we use the DeepSORT tracking architecture. DeepSORT builds upon the SORT algorithm by adding a neural network architecture that maintains deep appearance descriptors to track subjects across multiple frames [8]. Both YOLO and DeepSORT are discussed in detail in section 3, including their function, architecture, and how we will use them in our project.

2.1. Drone Computer Mount Design

We have two different computers flying the drone: the **Flight Controller (FC)** and the **Companion Computer**. The flight controller manages the drone's motors, sensors, communication, and feedback loops, enabling stable and level flight, and it is already wired and mounted to the drone's frame. The companion computer runs the computer vision pipeline and sends movement commands to the FC to follow the detected object. For this role, we use the **Radxa Dragon Q6A** single-board computer (SBC), which provides 12 TOPs of AI compute and supports the **Qualcomm AI Runtime (QAIRT)** for efficient model deployment. The companion computer is paired with a **Radxa IMX219** camera, which captures the video feed used for object detection.

Because the drone's frame has limited space, we designed a custom mounting solution in **Fusion 360**. The Radxa Dragon Q6A shares the same form factor as a Raspberry Pi 5 (85mm x 55mm), so we created a baseplate with protective walls and legs that allow the mount to be strapped securely to the drone while maintaining access to the IO ports. To mount the camera, we took inspiration from the modular design of a GoPro mount. We designed a 3-prong GoPro-style base mount that attaches to the baseplate, along with a protective camera enclosure that includes two prongs for adjustable attachment. The companion computer and camera mount are secured to the baseplate with nuts and

screws, creating a compact, adjustable mounting system for the drone.

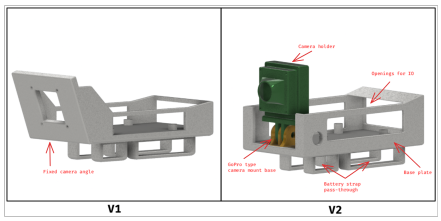

Figure 1. Computer Mount 3D Model

After finishing the 3D designs, we **3D-printed** the baseplate and camera mounts. The baseplate was printed in TPU to absorb any vibrations and save the companion computer in case of a crash.

3. Method

Our solution is divided into two major parts: object detection/tracking and generating commands for the drone to follow the object. The details of each part of our solution are as follows:

3.1. YOLO

To detect the subject our drone will follow, we will use the **YOLO (You Only Look Once)** object detection model. We choose YOLO over other models because it performs object detection using a single neural network pass. Other object detection algorithms, like R-CNN, operate in a two-stage process: first generating candidate regions that could contain objects, and then classifying each region individually. This two-stage approach increases computational cost and latency. YOLO treats object detection as a single regression problem, directly predicting object locations and class probabilities from the entire image in one forward pass through the network.

YOLO is made up of a deep convolutional neural network, which first subdivides the image into sections, then predicts **bounding boxes** and **classifications** for each subsection [3]. The network simultaneously learns spatial and object-classification features, enabling it to identify objects and estimate their locations quickly. YOLO uses filtering techniques like **non-maximum suppression** to eliminate duplicate bounding boxes while retaining the most confident detections. This method generates the final set of detected objects in the frame, each with a bounding box, class label, and confidence score.

The YOLO architecture (Fig. 2) has 24 convolutional layers for feature extraction and 2 fully connected layers that aggregate those features to output bounding box coordinates, confidence scores, and class probabilities [7].

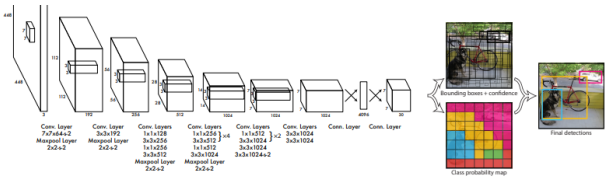

Figure 2. YOLO Architecture [7]

We will specifically use **YOLOv11** for our implementation because it provides strong real-time performance and is supported by Qualcomm AI Runtime (QAIRT). In our project, YOLO is responsible for detecting people in each frame captured by the drone's camera. While YOLO can identify our target, it cannot remember our target between frames. For this purpose, we use **DeepSORT** alongside it.

3.2. DeepSORT

After identifying our target, the system must maintain track of the specific individual across consecutive frames without switching to other people in the scene. To do this, we use **DeepSORT (Deep Simple Online and Realtime Tracking)** to track our subject over frames using the output of our YOLO object detection model. DeepSORT is a type of object tracker which can track multiple objects from different classes in a single frame, while maintaing high accuracy. Although our application focuses on a single target, using a multi-object tracker helps the system to keep the correct identify of the selected person even when other people come in the frame.

DeepSORT extends the earlier SORT (Simple Online and Realtime Tracking) algorithm. SORT is a lightweight tracking framework that uses a **Kalman Filter** to predict the next position of detected objects based on their past motion [2]. The Kalman filter calculates the object's state (position, velocity, and bounding box size) and predicts where it will appear in the following frame. SORT uses the **Hungarian assignment algorithm** to match predicted tracks with new YOLO detections by minimizing a distance metric, often based on bounding box overlap (Intersection over Union) [2].

DeepSORT enhances SORT by incorporating a **deep appearance descriptor** to improve tracking robustness. In addition to Kalman filter motion prediction, DeepSORT uses a small neural network to extract visual information from each detected object. These characteristics describe the object's visual appearance (such as clothing color, texture, and shape) [8]. During the association step, DeepSORT uses **Mahalanobis distance** (which determines how well a detection fits the expected motion model) and appearance similarity to link detections to existing tracks [8]. This enables the tracker to re-identify the same individual despite tem-

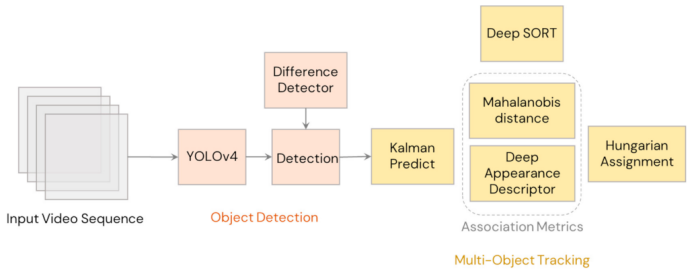

Figure 3. DeepSORT Architecture [6]

porary occlusions or missed detections. This capability is critical because it allows the drone to continue following the chosen subject even if YOLO temporarily fails to detect them or if they go behind obstacles.

DeepSORT assigns a unique **ID** to each identified object when it first appears in the frame, and that ID is retained across successive frames. For our project, we will lock onto the track ID with the maximum area covered in the frame (when the program starts) and follow only the object with that track ID.

3.3. ArduPilot

On the flight controller, we will be running **ArduPilot**, an open-source autopilot software that provides stable, autonomous flight control [1]. ArduPilot has several flight modes, but we will use **Guided Mode**, which allows an external computer to control the drone's position and velocity in real time. In Guided Mode, commands will be received over a serial connection.

ArduPilot runs a sensor fusion pipeline that continuously reads data from three hardware sources: an **IMU** (accelerometer and gyroscope), a **compass** (magnetometer), and a **GPS receiver**. These provide an accurate estimate of the drone's position, velocity, and orientation, allowing the drone to execute velocity commands without drifting. The flight controller can then convert incoming velocity setpoints into individual motor throttle values.

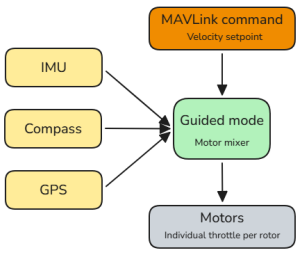

Figure 4. ArduPilot flight controller Architecture

3.4. MavLink

For communication between the Radxa Dragon Q6A and the ArduPilot flight controller in our system, we will use **MavLink** (**Micro Air Vehicle Link**). It is a binary serialization protocol designed for communication between unmanned vehicles and ground control stations [5]. For object tracking, we use a specific MAVLink message: **SET_POSITION_TARGET_LOCAL_NED**. This message allows the computer to send a velocity vector (v_x, v_y, v_z) in the drone's local NED (North-East-Down) coordinate frame. Velocity control lets the vision pipeline directly translate pixel-space tracking error into drone movement. The control logic computes the pixel offset (e) of the tracked target from the image center, scales it by a proportional gain (K_p), and sends the resulting velocity command to the flight controller:

$$e_x = x_{\text{target}} - x_{\text{center}}, \quad v_x = K_p \cdot e_x$$

This loop runs continuously, so the drone smoothly steers toward the target in real time.

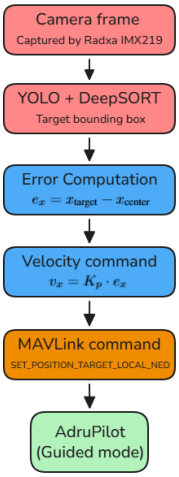

Figure 5. MAVLink control pipeline

3.5. Maintaining Distance between Drone and Object

Our goal is to keep the drone at a roughly constant distance from the subject. To control the drone's forward and backward movement relative to the tracked subject, we use the target's bounding box area as a proxy for distance. As the subject moves closer to the drone, the bounding box grows larger; as they move farther away, it shrinks. It will require some testing to determine the min and max thresholds for this system. We are still exploring other methods for measuring distance, such as using a depth estimation model. We are open to testing other distance-estimation methods and will select the best for our use case.

4. Experiments

We ran **YOLO11** on our systems to evaluate the model's performance. We used PyTorch, OpenCV, and Ultralytics to run the YOLO model. We downloaded stock footage of a drone following a person and ran the YOLO11 model on it [4]. Note that the model was run on our computers, with a GPU, and not on the Radxa Dragon Q6A. The results were accurate, with a tight bounding box and high confidence (Fig. 6).

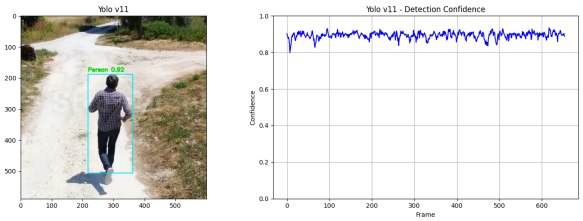

Figure 6. YOLO11 Results

Next, we ran DeepSORT on the same video. The goal was to see whether DeepSORT can retain a person's ID across multiple frames. We also ran the ByteTrack algorithm, which relies on motion (IoU matching) and high/low-score detection association [9]. We compared the performance of DeepSORT and ByteTrack (Fig. 7). ByteTrack had a higher average **confidence** than DeepSORT (Fig. 8). ByteTrack had a higher **FPS** than DeepSORT because DeepSORT uses a neural network and supports reidentification of tracked objects (Fig. 9). Both ByteTrack and DeepSORT were able to **maintain the track ID** of the subject.

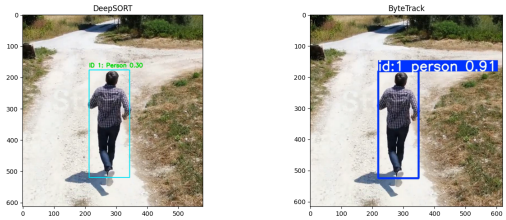

Figure 7. DeepSORT vs. ByteTrack

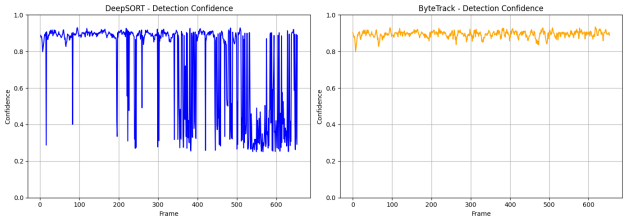

Figure 8. DeepSORT vs. ByteTrack Confidence

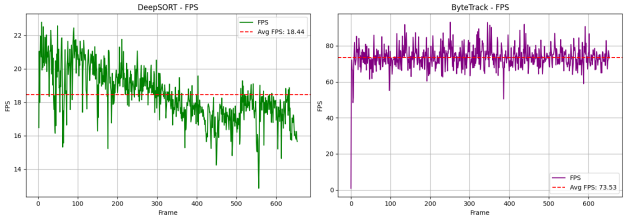

Figure 9. DeepSORT vs. ByteTrack FPS

Looking at these results, ByteTrack seems more appealing than DeepSORT, especially given its speed and confidence. But there is a tradeoff. ByteTrack does not support subject reidentification. If a subject is temporarily blocked/occluded from the frame, ByteTrack will generate a new ID for that subject. Whereas DeepSORT will most likely retain that specific track ID. So we have to decide whether we want speed or track ID consistency. Currently, we are leaning towards DeepSORT, but are open to trying out other tracking solutions.

5. Next Steps

1. **Collecting Training Data** - we will fly the drone and record a video of a person moving within the camera's frame. The collected footage will provide realistic training data containing variations in lighting, background, distance, and motion.

2. **Manually Annotating the Data** - we will manually identify the target person in each frame. Image labeling will be done using Label Studio, which allows bounding boxes to be drawn and labeled around the person in each frame.

3. **Fine-Tune Detection Model and Verify Results** - the annotated dataset will be used to fine-tune the YOLO model, thereby improving object detection performance from an aerial perspective. The fine-tuned model will be evaluated using standard object detection metrics, including precision, recall, and mean average precision (mAP).

4. **Software in the Loop (SITL) and Gazebo** - we will configure a simulation environment using ArduPilot SITL and Gazebo to test our code before running it on the drone. We will simulate a person inside Gazebo and pass a GStreamer pipeline to test our tracking code.

5. **Real-World Testing** - after sufficient testing in the simulator, we will deploy our code on the drone and test it physically. We will assess the drone's tracking/following accuracy and performance.

References

[1] Ardupilot copter — copter documentation. https://ardupilot.org/copter. 3

[2] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, page 3464–3468. IEEE, Sept. 2016. 2

[3] Zoumana Kelta. YOLO Object Detection Explained. https://www.datacamp.com/blog/yolo-object-detection-explained, 2 2026. 2

[4] Lenslogic. Drone following man. https://www.storyblocks.com/video/stock/video-from-above-of-a-man-wearing-jeans-a-plaid-shirt-and-sunglasses-man-running-along-a-country-road-dron-pursues-follow-a-running-man-shooting-from-the-drone-sk4c88rjsjxag2ibo. 4

[5] Mavlink interface — dev documentation. https://ardupilot.org/dev/docs/mavlink-commands.html. 3

[6] Addie Ira Borja Parico and Tofael Ahamed. Real time pear fruit detection and counting using yolov4 models and deep sort. *Sensors*, 21(14), 2021. 3

[7] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2016. 1, 2

[8] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric, 2017. 1, 2

[9] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box, 2022. 4